Projektowanie aplikacji webowych opartych o mikrousługi

LAB1 — Od problemu do własnej mikrousługi

Projektowanie aplikacji webowych opartych o mikrousługi

Czas: 90–120 minut zajęć; przygotowanie środowiska przed zajęciami, ewentualne dokończenie testów i README po zajęciach (około 30–60 minut)

Technologia: Python + FastAPI

Forma: praca indywidualna lub w parach

Rezultat: działająca pierwsza usługa własnego projektu, testy i uzasadnienie decyzji

1. Cel laboratorium

Po laboratorium powinieneś:

- rozumieć, czym jest mikrousługa,
- odróżniać mikrousługę od modułu, monolitu i modularnego monolitu,
- umieć wyznaczyć odpowiedzialność jednej usługi,
- zaprojektować proste API HTTP/REST,
- uruchomić usługę na własnym porcie,
- przechować zmianę między żądaniami i wyjaśnić utratę danych po restarcie,
- walidować dane wejściowe,
- zaimplementować i sprawdzić jedną regułę biznesową,
- obsługiwać błędy,
- przetestować API,
- wyjaśnić i obronić własne decyzje projektowe.

Najważniejsze: nie chodzi o przepisanie kodu. Kod jest środkiem do rozwiązania problemu.

Powiązanie z efektami kształcenia

Efekt	Co ćwiczysz na LAB1	Dowód wykonania
EK_03 — implementacja funkcjonalności z wykorzystaniem dokumentacji	Własna usługa, reguła biznesowa, obsługa błędów i testy	Działający kod, testy oraz dwa wykorzystane fragmenty dokumentacji opisane w README
EK_04 — projektowanie i komunikacja kilku usług	Pierwszy etap: podział własnego systemu i szkic przyszłej komunikacji	Dwie planowane usługi, właściciele danych i jeden kontrakt komunikacji; implementujesz teraz tylko pierwszą usługę
EK_01 i EK_02 — pojęcia oraz narzędzia	Porównanie architektur i rozróżnienie FastAPI, Uvicorna oraz klienta HTTP	Krótkie uzasadnienie architektury i samodzielne uruchomienie

LAB1 realizuje pierwszy temat laboratoryjny: podstawowa mikrousługa, API, porty i przechowywanie danych. Pełne EK_04 będzie weryfikowane w kolejnych etapach projektu. EK_05 oraz wiedza z EK_01 i EK_02 są również rozwijane i weryfikowane na wykładzie. DDD wprowadzamy przez pojęcia domeny, jej reguły i granice odpowiedzialności; pełne modelowanie jest tematem dalszych zajęć.

1.1. Minimum teorii przed rozpoczęciem kodowania

Mikrousługa

Mikrousługa to niezależnie uruchamiana i wdrażana część systemu, która:

- ma jasno określoną odpowiedzialność biznesową,
- udostępnia interfejs, na przykład HTTP/REST,
- działa jako osobny proces lub kontener,
- jeśli współpracuje z innymi usługami, komunikuje się przez sieć,
- może posiadać własne dane i być rozwijana niezależnie.

Moduł a mikrousługa

```
Moduł:
- działa wewnątrz jednej aplikacji,
- komunikuje się przez wywołania funkcji,
- zwykle jest wdrażany razem z całością.

Mikrousługa:
- działa jako osobna aplikacja,
- komunikuje się przez HTTP, RPC albo komunikaty,
- może być wdrażana i restartowana niezależnie.
```

Sam podział kodu na foldery nie tworzy mikroserwisów.

Osobny port także nie wystarcza: znaczenie mają spójna odpowiedzialność, własność danych i możliwość niezależnego wdrożenia. Na LAB1 budujesz pierwszą samodzielną usługę przyszłego systemu; nie uruchamiasz jeszcze całej architektury rozproszonej.

Czy mikroserwisy są zawsze lepsze?

Nie. Mały system może lepiej działać jako monolit albo modularny monolit. Mikroserwisy mają sens wtedy, gdy korzyści z niezależnego rozwoju, wdrażania i izolowania awarii uzasadniają dodatkową złożoność komunikacji sieciowej.

Pytania kontrolne przed kodowaniem

W pierwszej części laboratorium pytania odnoszą się do wspólnego przykładu `Locker Service`. Nie musisz jeszcze wymyślać własnego tematu — zrobisz to w części B.

```
Temat systemu: system paczkomatów
Planowana nazwa usługi: Locker Service
Jednozdaniowy opis usługi: obsługuje paczkomaty i ich skrytki
```

Dla `Locker Service` odpowiedz:

1. Za jaki konkretny problem biznesowy odpowiada `Locker Service`?
2. Za co nie powinna odpowiadać?
3. Czy jej odpowiedzialność jest wystarczająco spójna, aby wydzielić ją z systemu?
4. Jakie dane będą należeć do tej usługi?
5. Jakie informacje inne części systemu będą pobierać przez jej API?
6. Co użytkownik będzie mógł zrobić za pośrednictwem tej usługi?
7. Co stanie się z tą funkcją, jeśli proces usługi przestanie działać?
8. Czy dla tego problemu mikroserwis ma uzasadnienie, czy wystarczyłby moduł monolitu?

1.2. Ten sam system w różnych architekturach

Dla systemu paczkomatów można zaprojektować kilka poprawnych wariantów.

Wariant A — aplikacja monolityczna

```
Klient
  |
  v
Jedna aplikacja
├── obsługa paczkomatów
├── obsługa paczek
├── śledzenie
├── powiadomienia
└── użytkownicy
  |
  v
Jedna baza danych
```

Wszystkie funkcje działają w jednym procesie. Wywołanie funkcji odbywa się bezpośrednio w kodzie, np. moduł paczek może wywołać metodę modułu skrytek.

Wariant B — modularny monolit

```
Klient
  |
  v
Jedna aplikacja i jeden proces
├── moduł paczkomatów
├── moduł paczek
├── moduł śledzenia
├── moduł powiadomień
└── moduł użytkowników
  |
  v
Jedna baza danych
```

Moduły mają wyraźne granice i osobny kod, ale nadal są uruchamiane i wdrażane razem. Komunikacja odbywa się zwykle wewnątrz procesu.

Wariant C — mikroserwisy

```
Klient
  |
  v
API Gateway
  |
  +--> Locker Service       --> Locker DB
  +--> Parcel Service       --> Parcel DB
  +--> Tracking Service     --> Tracking DB
  +--> Notification Service --> Notification DB
```

Każda usługa może działać jako osobny proces, mieć własny port, własne wdrożenie i własne dane. Komunikacja odbywa się przez HTTP albo brokera wiadomości.

Ten sam przypadek biznesowy

Kurier umieszcza paczkę w skrytce.

W monolicie:

```
ParcelModule.placeParcel()
  ├── LockerModule.occupyCompartment()
  ├── TrackingModule.recordEvent()
  └── NotificationModule.sendMessage()
```

W mikroserwisach:

```
Parcel Service
      ├── HTTP: zajmij skrytkę ──> Locker Service
      │ <── potwierdzenie ───────┘
      │
      └── po potwierdzeniu: zdarzenie ParcelPlaced ──> RabbitMQ
                                                     ├──> Tracking Service
                                                     └──> Notification Service
```

`Locker Service` odpowiada za stan skrytki, a `Parcel Service` za operację dotyczącą paczki i publikację `ParcelPlaced`. To szkic docelowego przepływu, nie zadanie implementacyjne LAB1. Obsługę częściowych awarii i niezawodną publikację zdarzeń omówimy później.

Najważniejsza różnica

Cecha	Monolit	Modularny monolit	Mikrousługi
Uruchomienie	jeden proces	jeden proces	wiele procesów lub kontenerów
Komunikacja	wywołania w kodzie	wywołania modułów	HTTP, RPC lub komunikaty
Wdrożenie	całość razem	całość razem	usługi niezależnie
Baza danych	zwykle wspólna	zwykle wspólna	preferowana osobna dla usługi
Awaria procesu	może wyłączyć całość	może wyłączyć całość	wyłącza daną instancję; skutki mogą objąć zależne usługi
Złożoność początkowa	mała	średnia	duża

W tych porównaniach monolit i modularny monolit oznaczają aplikacje działające w jednym procesie. Izolacja awarii w mikroserwisach wymaga odpowiedniego zaprojektowania zależności; sam podział na procesy nie gwarantuje dostępności całego systemu.

---

2. Zasady pracy

Laboratorium ma dwie części:

1. krótki przykład wspólny — `Locker Service` dla systemu paczkomatów,
2. obowiązkowa praca własna — samodzielnie wymyślony temat i własna mikrousługa.

Przykład wspólny ma pokazać technikę. Własny projekt ma sprawdzić, czy potrafisz zastosować ją bez kopiowania gotowego modelu domenowego.

Pełne rozwiązanie referencyjne jest dostępne do analizy. Prowadzący pokazuje wybrane fragmenty, a student buduje własną usługę ze startera. Nie trzeba odtwarzać całego `Locker Service` przed rozpoczęciem własnego projektu.

Organizacja pracy

Przygotuj środowisko z sekcji 3 przed zajęciami. Proponowany podział 120 minut: 10 minut pojęć, 15 minut przykładu, 15 minut projektu domeny i API, 45 minut implementacji, 25 minut testów oraz 10 minut podsumowania. Przy bloku 90 minut dopracowanie testów i dokumentacji wykonaj po zajęciach. Termin oddania wskazuje prowadzący; komplet wymagań obowiązuje niezależnie od długości bloku.

Pracując w parze, zamieniajcie się rolami osoby piszącej i osoby sprawdzającej kod. W README zapiszcie wkład każdej osoby. Każda osoba samodzielnie wyjaśnia rozwiązanie i wykonuje drobną zmianę podczas odbioru.

Cały projekt semestralny wymaga co najmniej dwóch różnych mikrousług na osobę oraz pozostałych wymaganych komponentów. Jedna wspólna usługa na LAB1 jest dopuszczalnym etapem pracy w parze, ale nie wyczerpuje indywidualnego wymagania semestralnego. Własną domenę będziesz rozwijać również na kolejnych laboratoriach.

Korzystanie z AI

Możesz korzystać z AI do:

- wyjaśnienia błędu,
- zaproponowania wariantów API,
- wygenerowania szkicu,
- przygotowania testów,
- refaktoryzacji kodu.

Musisz jednak:

- uruchomić i zweryfikować kod,
- wykonać testy,
- rozumieć istotne fragmenty,
- umieć wyjaśnić swoje decyzje,
- wpisać w README, w jaki sposób użyto AI.

Kod, którego nie potrafisz uruchomić ani wyjaśnić, nie jest poprawnym rozwiązaniem.

---

3. Przygotowanie środowiska

Użyj Pythona 3.12 lub nowszego. Polecenia poniżej są przeznaczone dla PowerShella. Sprawdź wersję:

```
python --version
```

Pobierz starter ZIP i rozpakuj go. Otwórz terminal w katalogu zawierającym `requirements.txt`, `app` i `tests`. Możesz też skopiować katalog lab1-template poza katalog materiałów i nadać kopii własną nazwę.

Utwórz środowisko i zainstaluj zależności:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Nie musisz aktywować środowiska: jawna ścieżka do jego Pythona działa także wtedy, gdy PowerShell blokuje `Activate.ps1`.

Starter ma już `app/main.py` z endpointem `/health`. Uruchom go:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8101
```

Pozostaw serwer w pierwszym terminalu. W drugim sprawdź `Invoke-RestMethod http://localhost:8101/health`, a w przeglądarce otwórz dokumentację API. Starter zwraca nazwę `replace-me` — zmienisz ją na nazwę własnej usługi. Serwer zatrzymasz przez `Ctrl+C`.

Z katalogu projektu uruchom też test startowy:

```
.\.venv\Scripts\python.exe -m pytest -q
```

`app.main:app` wskazuje obiekt `app` w pliku `app/main.py`. Port `8101` wybiera lokalny adres usługi. `--reload` restartuje proces po zmianie kodu i służy do pracy programistycznej; przy danych w pamięci taki restart usuwa wprowadzone zmiany. Dla przykładu wspólnego używamy `8000`, dla własnej usługi np. `8101`. Porty muszą być różne dla procesów nasłuchujących na tym samym adresie na jednym komputerze; studenci na różnych komputerach mogą używać tego samego numeru.

FastAPI a Uvicorn

FastAPI jest frameworkiem, w którym definiujemy aplikację, endpointy, modele danych i walidację.

Uvicorn jest serwerem ASGI, który uruchamia aplikację FastAPI i nasłuchuje na określonym adresie oraz porcie. Sam plik Python nie wystarczy — potrzebujemy procesu serwera, który będzie przyjmował żądania HTTP.

▼ Materiał pomocniczy: ASGI, uruchomienie i diagnostyka portu

Co oznacza ASGI?

ASGI (*Asynchronous Server Gateway Interface*) to standard komunikacji pomiędzy serwerem uruchamiającym aplikację Pythona a samą aplikacją webową.

```
Przeglądarka / klient HTTP
            |
            v
Uvicorn — serwer ASGI
            |
            v
FastAPI — aplikacja ASGI
```

ASGI określa, w jaki sposób serwer przekazuje aplikacji:

- żądania HTTP,
- informacje o połączeniu,
- dane przesłane przez klienta,
- sygnały zakończenia żądania.

W drugą stronę aplikacja przekazuje:

- kod statusu,
- nagłówki,
- treść odpowiedzi.

ASGI jest następcą starszego standardu WSGI i został zaprojektowany z myślą o obsłudze kodu asynchronicznego. Umożliwia również obsługę WebSocketów i zdarzeń związanych z uruchamianiem oraz zamykaniem aplikacji.

Na tym laboratorium wystarczy zapamiętać:

> FastAPI tworzy aplikację zgodną z ASGI, a Uvicorn jest serwerem, który tę aplikację uruchamia i udostępnia przez HTTP.

Jeżeli mamy plik:

```
app/main.py
└── app = FastAPI(...)
```

uruchamiamy go poleceniem:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8101
```

Zapis `app.main:app` oznacza:

```
app.main  — moduł, czyli plik app/main.py
app       — obiekt aplikacji FastAPI znajdujący się w tym module
```

Parametr `--port 8101` wybiera port, na którym działa usługa. Parametr `--reload` powoduje automatyczne ponowne uruchomienie serwera po zapisaniu zmian w kodzie. Jest wygodny podczas nauki, ale nie powinien być używany jako ustawienie produkcyjne.

W przykładzie wspólnym używamy portu `8000`; własny projekt powinien używać innego portu, np. z zakresu `8101–8199`.

Jak sprawdzić port krok po kroku?

Po uruchomieniu powyższego polecenia nie zamykaj terminala — działający proces zajmuje port.

W terminalu powinien pojawić się komunikat podobny do:

```
Uvicorn running on http://127.0.0.1:8101
```

W drugim terminalu sprawdź endpoint zdrowia:

```
Invoke-RestMethod http://localhost:8101/health
```

Jeśli usługa działa, otrzymasz odpowiedź JSON, np.:

```
status service
------ -------
ok     my-service
```

Możesz też otworzyć dokumentację API w przeglądarce:

```
http://localhost:8101/docs
```

Sprawdzenie, czy port jest zajęty

```
Get-NetTCPConnection -LocalPort 8101 -State Listen
```

Jeżeli polecenie zwróci rekord, na porcie działa jakiś proces. Identyfikator procesu znajduje się w kolumnie `OwningProcess`.

Możesz sprawdzić nazwę procesu:

```
Get-Process -Id <PID>
```

Zastąp `<PID>` numerem z poprzedniego polecenia.

Jeżeli port jest zajęty, uruchom usługę na innym porcie:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8102
```

Własny port sprawdza się więc na trzy sposoby:

1. komunikat Uvicorna po uruchomieniu,
2. odpowiedź z `http://localhost:8101/health`,
3. obecność procesu nasłuchującego przez `Get-NetTCPConnection`.

Brak rekordu (lub komunikat o braku pasujących obiektów) oznacza, że polecenie nie znalazło nasłuchującego procesu na tym porcie.

---

4. Część A — przykład wspólny

Domena

System paczkomatów może mieć między innymi następujące usługi:

```
Locker Service       — paczkomaty i skrytki
Parcel Service       — paczki i ich statusy
Tracking Service     — historia zdarzeń
Notification Service — powiadomienia
Delivery Service     — transport i dostawy
```

W tej części analizujemy tylko `Locker Service`.

Odpowiedzialność usługi

`Locker Service`:

- przechowuje paczkomaty,
- przechowuje skrytki,
- informuje o statusie skrytki,
- pozwala dodać paczkomat,
- pozwala zmienić status skrytki.

Nie zajmuje się:

- kontami użytkowników,
- płatnościami,
- historią przesyłek,
- wysyłaniem wiadomości,
- transportem paczek.

Model skrytki

```
id         — identyfikator skrytki
locker_id  — identyfikator paczkomatu
size       — S, M albo L
status     — free, reserved, occupied albo out_of_service
```

API z przykładu

Metoda	Endpoint	Znaczenie
GET	`/health`	Sprawdzenie działania usługi
GET	`/lockers`	Lista paczkomatów
GET	`/lockers/{locker_id}`	Jeden paczkomat
POST	`/lockers`	Dodanie paczkomatu
GET	`/lockers/{locker_id}/compartments`	Lista skrytek
POST	`/lockers/{locker_id}/compartments`	Dodanie skrytki — rozszerzenie przykładu
PATCH	`/compartments/{id}/status`	Zmiana statusu skrytki

Przykład przechowuje dane w pamięci. Przy starcie tworzy paczkomat `KRK-1024` i dwie wolne skrytki. Reguła biznesowa określa przejścia: `free → reserved → occupied → free` oraz `free ↔ out_of_service`. Ustawienie aktualnego statusu ponownie jest dozwolone i nie zmienia stanu. Inne przejście zwraca `409`, nieznana wartość statusu — `422`, a brak skrytki — `404`.

Uruchomienie rozwiązania referencyjnego

W osobnym terminalu, zaczynając od katalogu głównego materiałów, przejdź do rozwiązania referencyjnego:

```
cd 2026_2027/lab1/lab1-reference/locker-service
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

Otwórz:

- http://localhost:8000/docs
- http://localhost:8000/redoc

Sprawdź:

```
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod http://localhost:8000/lockers
Invoke-RestMethod http://localhost:8000/lockers/KRK-1024/compartments
```

Przetestuj też:

```
curl.exe -i http://localhost:8000/lockers/UNKNOWN
```

Zwróć uwagę, że nieistniejący zasób kończy się kodem `404`, a nie pustą odpowiedzią `200`.

`curl.exe -i` pokazuje także status i nagłówki odpowiedzi. Jeśli nie masz `curl.exe`, wykonaj żądanie w `/docs` i odczytaj pole „Code”. Nie używamy skrótu `curl`, którego znaczenie może zależeć od wersji PowerShella.

W `/docs` wykonaj `PATCH` zmieniający status `KRK-1024-L1` z `free` na `reserved`, a następnie osobnym `GET` sprawdź skrytki. Spróbuj zmiany `reserved → out_of_service`: oczekuj `409` i sprawdź ponownym `GET`, że status nadal wynosi `reserved`. Instrukcję pełnego scenariusza i testów znajdziesz w README rozwiązania referencyjnego.

Pytania po przykładzie

Odpowiedz krótko:

1. Dlaczego skrytki należą do `Locker Service`?
2. Dlaczego historia przesyłek nie powinna być przechowywana w tej usłudze?
3. Co powinno się stać, gdy paczkomat nie istnieje?
4. Czy `Locker Service` musi znać dane użytkownika?
5. Czy ta usługa jest już całym systemem paczkomatów?

---

5. Część B — wymyśl i zbuduj własną usługę

To jest główna część laboratorium.

Krok 1 — wybierz problem

Wybierz prostą, znaną domenę albo zaproponuj własną:

- rezerwacja sal,
- biblioteka,
- plan zajęć,
- zgłoszenia IT,
- wypożyczalnia rowerów,
- wydarzenia i bilety,
- platforma gier,
- inteligentny budynek,
- paczkomaty.

Nie tworzysz całej aplikacji. Wybierasz jeden ograniczony obszar.

Przykład:

```
Temat: system uczelniany
Usługa: Room Reservation Service
```

Krok 2 — określ granicę odpowiedzialności

Uzupełnij:

```
Nazwa usługi:

Usługa odpowiada za:
-
-
-

Usługa nie odpowiada za:
-
-
-
```

Jeśli nie potrafisz jasno opisać granicy, temat jest jeszcze niedopracowany.

Zaproponuj co najmniej dwie usługi przyszłego systemu. Teraz zaimplementujesz tylko pierwszą. Uzupełnij tabelę:

Usługa	Odpowiedzialność biznesowa	Własne dane	Operacje	Informacje potrzebne od innej usługi
Pierwsza — implementowana na LAB1	…	…	…	…
Druga — planowana na później	…	…	…	…

Wypisz 3–5 pojęć używanych w swojej domenie i krótko je zdefiniuj. Wskaż, kto może zmieniać poszczególne dane. Wyjaśnij, dlaczego podział wynika z odpowiedzialności biznesowej, a nie wyłącznie z tabel w bazie. Granica modelu i znaczenia jego pojęć jest punktem wyjścia do poznania *bounded context* w DDD; nie oznacza automatycznie osobnej usługi dla każdej encji.

Naszkicuj jedno przyszłe wywołanie HTTP między usługami: nadawca, odbiorca, metoda, ścieżka, przykładowe żądanie i odpowiedź oraz zachowanie przy niedostępności odbiorcy. To projekt kontraktu — nie implementuj jeszcze komunikacji ani sprawdzania danych w drugiej usłudze.

Narysuj trzy małe warianty własnego systemu: monolit, modularny monolit i mikrousługi. Wystarczą prostokąty i strzałki z podpisami „wywołanie w kodzie” lub „HTTP”. Zaznacz granice odpowiedzialności, procesy i właścicieli danych. Schematy umieść w README; wykorzystasz je w podsumowaniu.

Checkpoint przed kodowaniem: wyjaśnij prowadzącemu zakres pierwszej usługi, rolę drugiej i jedną regułę, za którą odpowiada pierwsza.

Krok 3 — zaprojektuj model danych

Zdefiniuj minimum jeden główny obiekt domenowy.

Przykład:

```
RoomReservation
- id
- room_id
- reserved_by
- starts_at
- ends_at
- status
```

Samodzielnie zdecyduj:

- które pola są wymagane,
- jakie są dozwolone wartości statusu,
- czy identyfikator nadaje klient, czy serwer,
- czy dane powinny być przechowywane w jednym, czy kilku modelach.

Zapisz co najmniej jedną regułę biznesową wykraczającą poza wymagane pola, typy i unikalność identyfikatora. Przykłady:

- rezerwacja kończy się po swoim początku,
- rezerwacje tej samej sali nie nakładają się,
- roweru w naprawie nie można wypożyczyć,
- zamknięte zgłoszenie nie może przejść bezpośrednio do stanu „w realizacji”.

Wybierz jedną regułę, podaj przykład dozwolonej i niedozwolonej operacji oraz oczekiwany kod błędu. Reguła ma dotyczyć danych należących do twojej usługi, żeby dało się ją sprawdzić bez uruchamiania drugiej usługi.

Krok 4 — zaprojektuj API

Zaprojektuj minimum **trzy endpointy domenowe oraz osobny `GET /health`**, czyli co najmniej cztery operacje API:

Metoda	Endpoint	Znaczenie	Sukces
GET	`/...`	odczyt listy	200
GET	`/.../{id}`	odczyt jednego elementu	200
POST albo PATCH	`/...` albo `/.../{id}`	utworzenie albo modyfikacja	odpowiednio 201 albo 200

Wybierz co najmniej jedną operację zapisu. Jeżeli wybierasz wyłącznie `PATCH`, przygotuj dane początkowe i wskaż modyfikowane pola. Przykładowe ścieżki zastąp nazwami własnych zasobów. `/health` zwraca status działania i nazwę usługi, bez danych domenowych.

Ustal również:

- jaki kod oznacza brak zasobu,
- jaki kod oznacza konflikt,
- jaki kod oznacza błędne dane,
- jak wygląda odpowiedź błędu.

Dla każdej operacji opisz pola i typy żądania, odpowiedź sukcesu oraz możliwe błędy. Dołącz przykładowy JSON. Przyjmij `404` dla nieistniejącego zasobu, `409` dla konfliktu z aktualnym stanem i `422` dla niepoprawnych danych. Rozróżnij pustą listę (`200` i `[]`) od brakującego elementu (`404`).

Możesz zachować standardowy format FastAPI: `{"detail": "opis"}` dla `HTTPException` i listę szczegółów w `detail` dla walidacji `422`. Opisz oba formaty zamiast deklarować identyczną strukturę wszystkich błędów. Sprawdź zgodność działającego API z tabelą i przykładami w README.

Krok 5 — utwórz aplikację

Minimalna struktura:

```
my-service/
├── app/
│   └── main.py
├── tests/
├── requirements.txt
└── README.md
```

Zacznij od zmiany nazwy usługi w starterze i sprawdzenia `/health`. Dodawaj kolejne elementy w małych krokach: model, odczyt, zapis, reguła biznesowa, błędy i testy.

Przechowuj dane w słowniku lub liście **w pamięci procesu**. Utwórz magazyn raz dla danej instancji aplikacji, a nie od nowa w każdym żądaniu. Zmiana wykonana przez `POST` lub `PATCH` musi być widoczna w następnym `GET`. Każda nowa instancja aplikacji powinna otrzymać własny magazyn; wykorzystaj funkcję `create_app()` ze startera, aby testy zaczynały od znanego stanu.

Na LAB1 nie potrzebujesz bazy danych, zapisu do pliku ani cache. Restart, również wywołany przez `--reload`, usuwa zmiany z pamięci i przywraca ewentualne dane początkowe. Dwie instancje uruchomione na różnych portach mają osobne magazyny. Te ograniczenia opisz w README.

Nie kopiuj modelu `Locker Service`. Możesz używać tego samego mechanizmu technicznego, ale domena, nazwy, dane i reguły muszą wynikać z twojego tematu.

Krok 6 — zaimplementuj walidację i błędy

Obsłuż minimum trzy przypadki:

- zasób nie istnieje,
- dane wejściowe są niepoprawne,
- występuje konflikt, np. duplikat albo kolizja terminu.

Przykładowe kody:

```
200 OK
201 Created
404 Not Found
409 Conflict
422 Unprocessable Entity
```

Nie wybieraj kodów mechanicznie. W README zapisz, dlaczego użyłeś danego kodu.

Zaimplementuj także regułę biznesową z kroku 3. Wszystkie warunki odrzucenia operacji sprawdź **przed zmianą danych**. Niepoprawna operacja nie może pozostawić częściowo zmienionego stanu. Naruszenie reguły może być jednym z wymaganych przypadków `409` lub `422`, zależnie od jej znaczenia; nie musi tworzyć czwartej kategorii błędu.

Praca z dokumentacją — wymaganie EK_03

Wykorzystaj co najmniej dwa konkretne fragmenty oficjalnej dokumentacji, np. modele żądań FastAPI, obsługę błędów albo testowanie. W README podaj odnośnik, problem, który rozwiązywałeś, zastosowane rozwiązanie i sposób jego sprawdzenia. Sam wykaz linków lub zapis rozmowy z AI nie wystarcza.

Krok 7 — napisz testy

Przygotuj powtarzalne scenariusze:

Scenariusz	Co należy wykazać
`/health`	`200` i poprawna nazwa usługi
Lista i szczegóły	`200`, właściwe pola i dane znanego zasobu
Zapis → odczyt	Poprawny `POST` / `PATCH`, a następnie osobny `GET` potwierdzający zapisaną zmianę
Brak zasobu	`404` i opis błędu
Niepoprawne dane	`422`, informacja o błędzie i brak zmiany magazynu
Konflikt	`409`, opis przyczyny i brak zmiany magazynu
Reguła biznesowa	Przykład dozwolony oraz niedozwolony; po odrzuceniu stan pozostaje bez zmian
Restart procesu	Zmiana znika, a dane początkowe wracają, jeśli je przewidziano

Reguła biznesowa może być sprawdzana w scenariuszu konfliktu lub niepoprawnych danych. Nie musisz powielać identycznego testu.

Napisz **co najmniej trzy własne testy automatyczne w `pytest`**: zapis i odczyt, błędne dane bez zmiany stanu oraz konflikt bez zmiany stanu. Co najmniej jeden z nich ma sprawdzać regułę biznesową; dodaj kolejny, jeśli wybrane trzy jej nie obejmują. Test `/health` dostarczony w starterze nie zalicza się do tych trzech.

W testach sprawdzaj zarówno kod HTTP, jak i istotne pola JSON. Każdy test rozpoczynaj od świeżej aplikacji i znanych danych, np. przez fixture wywołującą `create_app()`. Testy nie mogą zależeć od kolejności uruchomienia. `TestClient` nie wymaga działającego Uvicorna; osobno trzeba sprawdzić rzeczywisty port i restart procesu.

Pozostałe przypadki możesz sprawdzić ręcznie w `/docs`, Postmanie lub przez `curl.exe`. Dla każdego ręcznego scenariusza zapisz: stan początkowy, dokładne żądanie, oczekiwany status i treść, wynik rzeczywisty oraz sposób przywrócenia stanu. Zrzut ekranu bez tych informacji nie wystarcza.

Eksperyment z restartem: dodaj zasób lub zmień dane początkowe, potwierdź zmianę osobnym `GET`, zatrzymaj Uvicorna przez `Ctrl+C`, uruchom go ponownie i powtórz `GET`. Wyjaśnij, dlaczego dane przetrwały między żądaniami, ale nie między uruchomieniami. Nie edytuj kodu w trakcie eksperymentu, bo `--reload` może wykonać dodatkowy restart.

Krok 8 — przygotuj README

README musi zawierać:

- opis problemu,
- opis odpowiedzialności usługi,
- opis tego, za co usługa nie odpowiada,
- tabelę dwóch planowanych usług, pojęcia domeny i szkic przyszłego kontraktu komunikacji,
- trzy proste schematy architektury i odpowiedzi z sekcji 6,
- regułę biznesową z przykładem dozwolonym i niedozwolonym,
- instrukcję uruchomienia,
- listę endpointów,
- przykładowe żądania i odpowiedzi,
- opis błędów,
- opis testów,
- polecenie uruchomienia testów automatycznych oraz wynik eksperymentu z restartem,
- dwa wykorzystane fragmenty dokumentacji wraz z opisem zastosowania i weryfikacji,
- ograniczenia rozwiązania,
- informację o użyciu AI (albo „nie korzystano”) i wkład każdej osoby w parze.

---

6. Porównanie architektur

Na zakończenie odpowiedz własnymi słowami.

Pytanie	Monolit	Mikrousługi	Modularny monolit
Czy wszystkie funkcje są wdrażane razem?	Tak	Mogą być wdrażane niezależnie	Tak
Czy moduły komunikują się przez sieć?	Zwykle nie	Tak	Zwykle nie
Czy moduły/usługi działają w osobnych procesach?	Nie, przyjęliśmy jeden proces	Tak	Nie, przyjęliśmy jeden proces
Czy awaria procesu może objąć cały system?	Tak	Może dotknąć także zależnych usług	Tak
Czy dane mogą być rozdzielone?	Często wspólne	Zwykle rozdzielone	Mogą być logicznie rozdzielone
Czy wdrożenie jest niezależne?	Zwykle nie	Tak	Zwykle nie

Odpowiedz:

1. Dlaczego twoja usługa może być mikroserwisem?
2. Dlaczego nie jest tylko klasą albo modułem?
3. Jak wyglądałby twój system jako monolit?
4. Co zmieniłoby się po podzieleniu go na mikroserwisy?
5. Jakie koszty wprowadza komunikacja sieciowa?
6. Czy w twoim przypadku mikroserwisy są rzeczywiście uzasadnione?

Poprawną odpowiedzią może być również stwierdzenie, że dla małego systemu lepszy byłby monolit lub modularny monolit.

---

7. Checkpointy i rozmowa techniczna

Przy odbiorze pokaż poniższe elementy. Przy pracy w parze każda osoba odpowiada indywidualnie.

Checkpoint A — granica

Student potrafi wyjaśnić, za co odpowiada usługa, jakie dane posiada i jaką rolę pełni planowana druga usługa.

Checkpoint B — API

Student potrafi wyjaśnić metodę, ścieżkę i kod odpowiedzi każdego endpointu.

Checkpoint C — błędy

Student pokazuje działające `404`, `422` i `409`, wyjaśnia regułę biznesową oraz wykazuje, że odrzucona operacja nie zmieniła danych.

Checkpoint D — testy

Student uruchamia testy, wyjaśnia ich warunki początkowe i wyniki oraz pokazuje zapis i odczyt przez HTTP. Potrafi wyjaśnić wynik restartu procesu.

Checkpoint E — własny kod

Student wskazuje zastosowane fragmenty dokumentacji oraz sposób weryfikacji pomocy AI, jeśli jej używał. Samodzielnie wykonuje drobną zmianę zadaną przez prowadzącego, np. dodaje ograniczenie długości pola i odpowiadający mu test. Zmiana powinna zajmować około 3–5 minut i dotyczyć zakresu LAB1.

---

8. Kryteria zaliczenia

Punktacja LAB1 służy ocenie tego etapu i informacji zwrotnej. Zachowuje podział 60 punktów za kod i 40 za dokumentację. Ocena końcowa przedmiotu dotyczy całego rozwijanego projektu według sylabusa; nie jest automatyczną średnią punktów z laboratoriów.

Obszar	Element	Maksimum
Kod	Uruchomienie, `/health`, lista, szczegóły i zapis widoczny w kolejnym odczycie	20
Kod	Model, walidacja, reguła biznesowa oraz poprawne `404`, `409`, `422` bez uszkodzenia stanu	20
Kod	Własne testy automatyczne, sensowne asercje i niezależność testów	20
Dokumentacja	Domena, granice i właściciele danych, druga usługa oraz przyszły kontrakt komunikacji	10
Dokumentacja	Powtarzalna instrukcja uruchomienia, kontrakt API, przykłady i błędy	10
Dokumentacja	Scenariusze testowe, wyniki i wyjaśnienie restartu oraz ograniczeń	10
Dokumentacja	Porównanie architektur, wykorzystanie dokumentacji, informacja o AI i autorach	10
Razem	**Kod 60 + dokumentacja 40**	**100**

W każdej pozycji punkty przyznaje się za poprawność i kompletność przedstawionych dowodów. Sam nagłówek w README albo test bez istotnych asercji nie potwierdza wykonania wymagania.

Zaliczenie etapu: co najmniej 50 punktów oraz spełnienie wszystkich warunków minimum:

- usługa działa na wskazanym porcie i ma `/health` oraz trzy wymagane operacje domenowe;
- zapis jest widoczny w osobnym odczycie, a student wyjaśnia utratę zmian po restarcie;
- działa jedna reguła biznesowa, walidacja i trzy wymagane kategorie błędów;
- odrzucone operacje nie zmieniają stanu, co potwierdzają testy;
- przechodzą co najmniej trzy własne testy automatyczne obejmujące wymagania kroku 7;
- README zawiera wymagane elementy i pozwala innej osobie odtworzyć uruchomienie oraz testy;
- każda osoba potrafi samodzielnie wyjaśnić kod i wykonać małą zmianę z checkpointu E.

Brak warunku minimum oznacza konieczność poprawy, nawet jeśli suma punktów przekracza 50. Termin poprawy ustala prowadzący.

---

9. Oddanie pracy

Oddaj repozytorium lub katalog zawierający:

```
my-service/
├── app/
├── tests/
├── requirements.txt
├── README.md
└── .gitignore
```

W następnym laboratorium własna usługa zostanie rozbudowana o trwałe przechowywanie danych i cache.

Nie dołączaj `.venv`, `__pycache__`, `.pytest_cache` ani sekretów. Przed oddaniem sprawdź instalację zależności i wykonanie testów w nowym środowisku. Starter jest pomocą techniczną; gotowy projekt musi zawierać twoją domenę, regułę i testy. W kwestii zakresu LAB1 obowiązuje ta instrukcja; przykład paczkomatów pokazuje jedną z możliwych implementacji.